全系统注入的方法

发布于：2016-6-20 18:07 | 449975次阅读 | 作者: 管理员 | 原作者: TP | 来自: 原创

1、概述

DLL注入技术: 是将一个Dll文件强行加载到目标进程中，比如把外挂dll模块注入到游戏进程，这样做的目的在于方便我们通过这个DLL读写目标进程指令或内存数据，（例如 HOOK游戏函数过程或篡改游戏内存数据实现外挂功能），或以被注入进程的身份去执行一些操作等。

全系统注入的优点：利用系统机制实现的全系统进程注入，可绕过比如游戏进程自身的防注入保护机制。比如远线程注入游戏可能会被拦截，但输入法注入，游戏很难拦截。

2、消息钩子注入

2.1 原理

Windows应用程序是基于消息驱动的。应用程序对各种消息响应从而实现各种功能。

消息钩子(Message Hook)是Windows消息处理机制的一个监视点，系统会自动将钩子安装到目标进程中达到监视指定类型消息的功能。也就是说通过SetWindowsHookEx 系统会自动将钩子dll注入到目标进程。

安装钩子的函数原型如下：

```
HHOOK SetWindowsHookEx(

  int idHook,  //钩子类型
```

```
HOOKPROC lpfn,
```

```
HINSTANCE hMod,
```

```
DWORD dwThreadId
```

```
);
```

其中dwThreadId为0时，则是全局钩子，即会注入dll到系统所有窗口进程，否则是线程钩子，即只能将dll注入到目标线程所属的进程。当全局钩子时，钩子处理过程HOOKPROC lpfn 必须位于dll中。

2.2 步骤和源码

源码将以WH_CBT 钩子为例，实现全系统dll注入。 computer-based training (CBT) 基于电脑的训练，比如创建窗口，移动窗口等操作的都会收到通知。

在以下事件之前，系统都会调用WH_CBT Hook子程，这些事件包括：

1. 激活，建立，销毁，最小化，最大化，移动，改变尺寸等窗口事件；

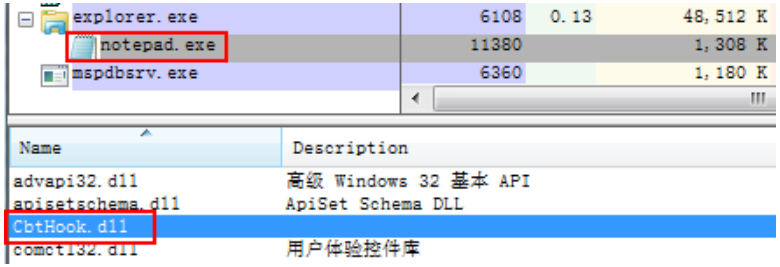
2. 完成系统指令；

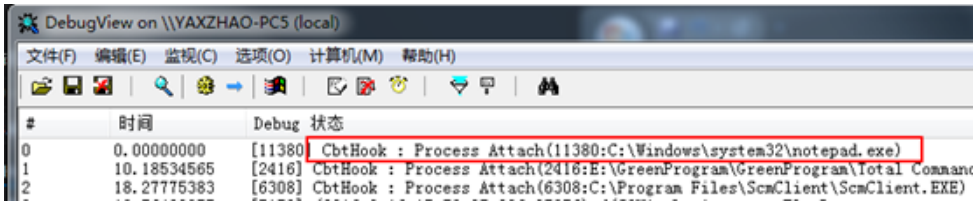
3. 来自系统消息队列中的移动鼠标，键盘事件；

4. 设置输入焦点事件；

5. 同步系统消息队列事件。

安装WH_CBT 钩子将CbtHook.dll注入到notepad进程中的效果：





1. 编写被注入的Dll和钩子处理过程

编写一个DLL，并且显式导出CBTProc ()钩子处理过程，主要代码如下：

```
LRESULT CALLBACK CBTProc(int nCode, WPARAM wParam, LPARAM lParam)

{

char szFileName[MAX_PATH];



if( (HCBT_CREATEWND == nCode) || (HCBT_ACTIVATE == nCode) )

{

memset(szFileName, 0, sizeof(szFileName));

::GetModuleFileNameA(NULL, szFileName, sizeof(szFileName));
```

```
DebugPrintA(0, "Code(%d) (%s)\n", nCode, szFileName);
}
return CallNextHookEx(g_hCBT, nCode, wParam, lParam);
}
```

2. 安装HOOK

编写一个exe，使用SetWindowsHookEx()向系统安装钩子，首先需要将HOOK的DLL 加载到exe本身的进程中，以此得到DLL的模块句柄，再使用GetProcAddress()得到DLL中显示导出的函数MyMessageProc()的函数地址，最后遍历出待注入进程的线程ID，这样SetWindowsHookEx()就可以利用这些参数进行HOOK了。主要代码如下图所示：

```
            if (NULL == g_hCbtHook)
g_hCbtHook = ::LoadLibrary(szPathName);


if (NULL == g_hCbtHook)
{
lResult = GetLastError();
::DebugPrintA(0, "%s : Load GlobalHook module '%s' fail(%d)\n", C_ModuleNameA, szPathName, lResult);
break;
}
```

```
CBTProc = (PCBTPROC)::GetProcAddress(g_hCbtHook, "CBTProc");
if (NULL == g_hCbtHook)
{
lResult = GetLastError();
::DebugPrintA(0, "%s : Get GlobalHook function fail(%d)\n", C_ModuleNameA, lResult);
break;
}

g_hCBT = SetWindowsHookEx(WH_CBT, CBTProc, g_hCbtHook, 0);
if (NULL == g_hCBT)
{
lResult = GetLastError();
::DebugPrintA(0, "%s : Set GlobalHook fail(%d)\n", C_ModuleNameA, lResult);
break;
}
```

3. 卸载钩子

利用LoadLibrary()得到的模块句柄把本身进程的DLL释放掉，代码如下所示：

FreeLibrary(g_hCbtHook);

消息钩子注入只熟悉SetWindowsHookEx()和DLL导出函数就可以很容编写，所以容易实现。

2.3 练习

使用WH_KEYBOARD键盘钩子注入到Notepad.exe，监视记事本键入的字符。

3、注册表注入

3.1 原理

注册表(Reg)注入原理是利用在Windows 系统中，当REG以下键值中存在有DLL文件路径时，会跟随EXE文件的启动加载这个 DLL文件路径中的DLL文件。

AppInit_Dlls注册表：

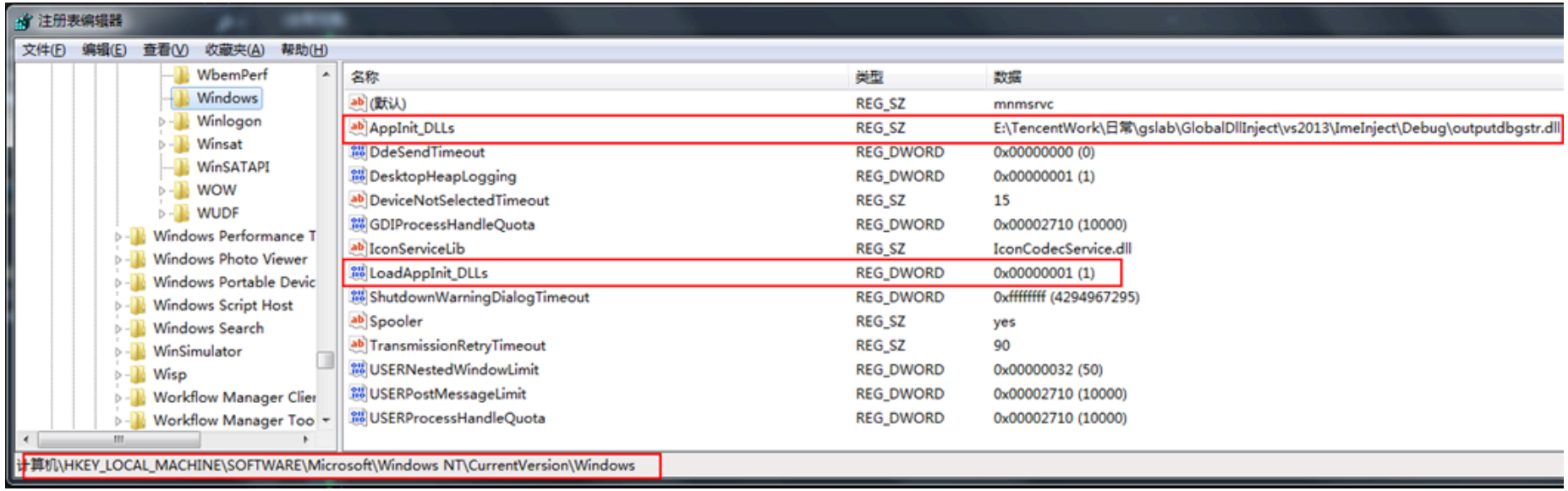
注册表项 HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows中有两个值：

LoadAppInit_Dlls：键值中指定要注入的DLL

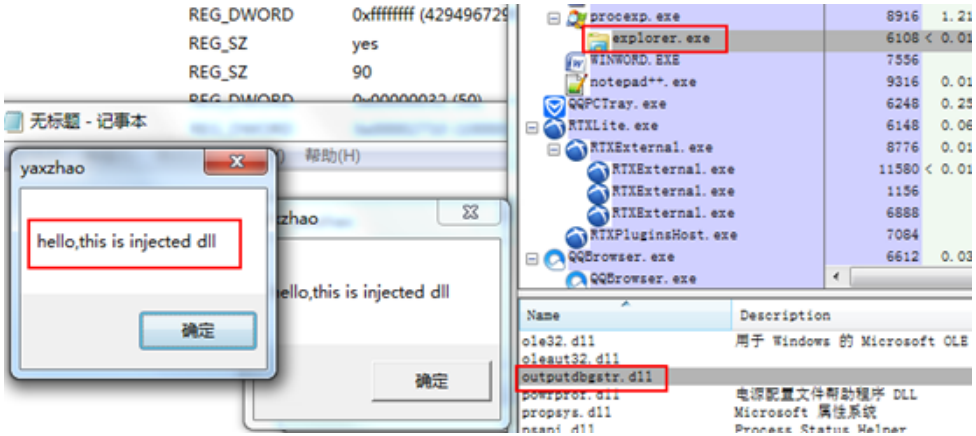
AppInit_Dlls：若其键值为1,则注入LoadAppInit_Dlls中指定的DLL，**若为0则不注入。**注:(1)LoadAppInit_Dlls中的值当如 果遇到有多个DLL文件时，需要用逗号或者空格隔开多个DLL文件的路径,所以**DLL的路径中最好不要有空格。**

使用范围：

任何加载User32.DLL的程序，user32.dll的DllMain会先尝试加载注册表项AppInit_Dlls中的DLL。因为所有的GUI应用程序在启动时都会加载User32.dll，因此这种方法会影响所有的GUI程序。



使用Process Explorer查看进程模块来确认目标dll是否被注入。



3.2 步骤和源码

需要解决的就是关于注册表操作的Windows API了，如下所示：

RegOpenKeyEx	打开注册表键值

RegQueryValueEx	查询键值
RegSetValueEx	设置键值
RegCloseKey	关闭键值

主要代码如下:

```
BOOL  AddRegItem(CHAR*  szInjectFilePath)
{
//打开键值
LSTATUS  nReg  =  ERROR_SUCCESS;
HKEY hKey;
CHAR       szRegPath[MAX_PATH]  =  "SOFTWARE\\Microsoft\\Windows  NT\\CurrentVersion\\Windows";


nReg  =  RegOpenKeyEx(
HKEY_LOCAL_MACHINE,
szRegPath,
0,
KEY_ALL_ACCESS,
&hKey);
```

```
if (nReg != ERROR_SUCCESS)
{
return FALSE;
}

//查询键值
DWORD dwReadType;
DWORD dwReadCount;
TCHAR szReadBuff[1024] = { 0 };
nReg = RegQueryValueEx(hKey,
_T("AppInit_DLLs"),
NULL,
&dwReadType,
(BYTE*)&szReadBuff,
&dwReadCount);
```

```
if (nReg != ERROR_SUCCESS)
{
return FALSE;
}

//若dll名称已经在内容中，则不用重复添加
if (StrStrI(szReadBuff,szInjectFilePath))
{
printf("dll already in reg=%s\n", szReadBuff);
return FALSE;
}

//原来已有内容就加入空格后再附加新dll串
if (0 != _tcscmp(szReadBuff, _T("")))
{
_tcscat_s(szReadBuff, _T(" "));
}
```

```
_tcscat_s(szReadBuff,  szInjectFilePath);


//1.把dll路径设置到注册表中
nReg  =  RegSetValueEx(hKey,
_T("AppInit_DLLs"),
0,
REG_SZ,
(CONST  BYTE*)szReadBuff,
(_tcslen(szReadBuff)  +  1)*sizeof(TCHAR));
//2.启动  注册表加载dll
BYTE  byEnable[4]  =  {  0x1  };
nReg  =  RegSetValueEx(hKey,
_T("LoadAppInit_DLLs"),
0,
REG_DWORD,
(CONST  BYTE*)byEnable,
```

```
    4);

    printf("RegSetValueEx  AppInit_DLLs  =  %s Result=%d\n",  szReadBuff,  nReg);

    }
```

3.3 练习

由于该注册表会注入系统所有的GUI进程，如果我们只对游戏进程感兴趣，那么如何处理，使得目标代码只在游戏进程中运行呢？

4、输入法注入

4.1 原理

输入法注入是利用当前程序需要输入字符时切换到指定输入法，Windows系统就会把这个输入法需要的Ime文件装载到当前进程中，由于这个Ime文件本质上只是个存放在C:\Windows\System32目录下的特殊的DLL文件，因此我们可以利用这个特性，在Ime文件中使用LoadLibrary()函数待注入的DLL文件。

4.2 步骤和源码

1、编写Ime文件

输入法的Ime文件其实就是个显式导出19个特殊函数的DLL文件。如下图所示:

```
EXPORTS
    ImeConversionList
    ImeConfigure
    ImeDestroy
    ImeEscape
    ImeInquire
    ImeProcessKey
    ImeSelect
    ImeSetActiveContext
    ImeSetCompositionString
    ImeToAsciiEx
    NotifyIME

    ImeRegisterWord
    ImeUnregisterWord
    ImeGetRegisterWordStyle
    ImeEnumRegisterWord

    UIWndProc
    StatusWndProc
    CompWndProc
    CandWndProc
```

1. ImeConversionList //将字符串或字符转换成目标字串

2. ImeConfigure //配置当前ime参数函数

3. ImeDestroy //退出当前使用的IME

4. ImeEscape //应用软件访问输入法的接口函数

5. ImeInquire //启动并初始化当前ime输入法

6. ImeProcessKey //ime输入键盘事件管理函数

7. ImeSelect //启动当前的ime输入法

8. ImeSetActiveContext //设置当前的输入处于活动状态

```
9. ImeSetCompositionString     //由应用程序设置输入法编码

10. ImeToAsciiEx                //将输入的键盘事件转换为汉字编码事件

11. NotifyIME                   //ime事件管理函数

12. ImeRegisterWord             //向输入法字典注册字符串

13. ImeUnregisterWord           //删除被注册的字符串

14. ImeGetRegisterWordStyle

15. ImeEnumRegisterWord

16. UIWndProc        //用户界面接口函数

17. StatusWndProc    //状态窗口注册函数

18. CompWndProc      //输入编码窗口注册函数

19. CandWndProc      //选择汉字窗口注册函数
```

如果想编写功能完整的输入法程序，那么这19个导出函数都需要仔细的研究，但是对于只想实现注入的我们，现在只需要对ImeInquire()有比较深的认识就可以了。ImeInquire()是启动并初始化当前Ime输入法函数，声明如下：

```
BOOL WINAPI ImeInquire(LPIMEINFO lpIMEInfo,LPTSTR lpszUIClass,LPCTSTR lpszOption)
```

第一个参数lpIMEInfo比较重要，用于输入对Ime输入法初始化的内容结构，如果这个结构填写错误，就会导致输入法不能正常运行。第二个参数是输入一个class类名，我们需要先使用RegisterClassEx()注册出一个窗口类。

1.1 初始化ImeInquire()

主要代码如下所示：

```
BOOL  WINAPI  ImeInquire(LPIMEINFO  lpIMEInfo,LPTSTR  lpszUIClass,LPCTSTR  lpszOption)

{

//  输入法初始化过程

        lpIMEInfo->dwPrivateDataSize  =  0;  //系统根据它为INPUTCONTEXT.hPrivate分配空
间



        lpIMEInfo->fdwProperty  =  IME_PROP_KBD_CHAR_FIRST  |

                                                IME_PROP_IGNORE_UPKEYS  |

  IME_PROP_END_UNLOAD;



        lpIMEInfo->fdwConversionCaps  =  IME_CMODE_FULLSHAPE  |

IME_CMODE_NATIVE;



        lpIMEInfo->fdwSentenceCaps  =  IME_SMODE_NONE;

        lpIMEInfo->fdwUICaps  =  UI_CAP_2700;

lpIMEInfo->fdwSCSCaps  =  0;

        lpIMEInfo->fdwSelectCaps  =  SELECT_CAP_CONVERSION;

        _tcscpy(lpszUIClass,CLSNAME_UI);    //  注意该输入法基本窗口类必须注册，否则输入
```

```
法不能正常运行

        return  TRUE;

    }
```

1.2 注册输入法窗口类

使用RegisterClassEx注册窗口类，主要代码如下：

```
//*************************************************************

// 基本输入法窗口UI类注册

//*************************************************************

BOOL  ImeClass_Register(HINSTANCE  hInstance)

{

        WNDCLASSEX  wc;

        //

        //  register  class  of  UI  window.

        //

        wc.cbSize                =  sizeof(WNDCLASSEX);

        wc.style                  =  CS_INPUTSTAR  |  CS_IME;
```

```
        wc.lpfnWndProc        =  UIWndProc;

        wc.cbClsExtra          =  0;

        wc.cbWndExtra          =  2  *  sizeof(LONG);

        wc.hInstance            =  hInstance;

        wc.hCursor               =  LoadCursor(  NULL,  IDC_ARROW  );

        wc.hIcon                   =  NULL;

        wc.lpszMenuName      =  (LPTSTR)NULL;

        wc.lpszClassName    =  CLSNAME_UI;

        wc.hbrBackground    =  NULL;

        wc.hIconSm               =  NULL;

        if(  !RegisterClassEx(  (LPWNDCLASSEX)&wc  )  )

                return  FALSE;

return  TRUE;

}
```

1.3 Ime的DllMain注册窗口，并加载要被注入的dll

主要代码如下：

```
BOOL  WINAPI  DllMain(HINSTANCE  hinstDLL,DWORD  fdwReason,LPVOID  lpvReserved)

{

      switch(fdwReason)

        {

            case  DLL_PROCESS_ATTACH:

      OutputDebugString(_T("ime  DLL_PROCESS_ATTACH"));

      MyLoadCilentDLLFun();

    if(!ImeClass_Register(hinstDLL))

    return  FALSE;      //  DLL加载时注册必须的UI基本窗口类

    break;

        }

return  true;

}
```

1.4 IME版本信息

编写DLL时需要注意,当作IME文件的Dll需要有版本信息,**Version资源中FILETYPE为VFT_DRV, FILESUBTYPE为VFT2_DRV_INPUTMETHOD,**否则调用ImmInstallIME安装时会失败。

2、安装输入法

安装输入法的基本逻辑就是将自己编写的输入法设置为系统默认输入法，这样系统中所有进程就会默认加载这个恶意输入法程序。

首先需要得到系统当前的默认的输入法，以便恢复时使用。然后需要将ime文件拷贝到C:\Windows\System32目录下，最后将装载成功后将自己的输入法设置成为默认输入法，主要代码如下：

```
HKL  ImmInstallI(CString  CurrentImeFile,CString  SymImeFile)

{

  HKL  MYhkl  =  0;

  HKL  oldhkl=0;//用于卸载本输入法的时候恢复

  SystemParametersInfo(

  SPI_GETDEFAULTINPUTLANG,

  0,

  &oldhkl,

  0

  );

  CopyFile(CurrentImeFile,SymImeFile,FALSE);//重写已存在的文件

  MYhkl  =  ImmInstallIME(SymImeFile,L"IME注入输入法");

          return  MYhkl;

}

void  MSetIMEIsDefInput(HKL  Immhkl)
```

```
{

BOOL  bDefault=FALSE;

Sleep(1000);

if  (ImmIsIME(Immhkl))

{

if(g_IsDefaultIME)

{

//bDefault=SystemParametersInfo(SPI_SETDEFAULTINPUTLANG,0,&Immhkl,SPIF_SENDCHANGE||SPIF_UPDATEINIFILE);  //设置为默认输入法

bDefault=SystemParametersInfo(SPI_SETDEFAULTINPUTLANG,0,&Immhkl,SPIF_SENDWININICHANGE);

}

}

}
```

3、编写卸载输入法

当不再需要注入时，我们就需要卸载输入法。卸载输入法时需要先判定系统当前的输入法不是其原有默认输入法，确认无误后将系统的默认输入法恢复后，再将恶意输入法卸载即可，主要代码如下：

```
::SystemParametersInfo(
```

```
            SPI_SETDEFAULTINPUTLANG,
            0,
            &m_retV,
            SPIF_SENDWININICHANGE);
    if  (UnloadKeyboardLayout(m_hImeFile))
    {
            MessageBox(_T("输入法卸载成功"));
            }
```

输入法注入的实现需要对输入法IME文件的生成有所了解，API使用较多，所以实现起来比较难，但是由于系统存在多个输入法，被注入进程很难判别当前是可信赖输入法还是用于注入的恶意输入法，所以难以阻止，大大提高了注入的几率。

5、三种方法优缺点对比

消息钩子：

编写简单。xp上可以注入dll到系统进程，但win7上，由于Session 0 隔离机制，不能注入系统进程。同时由于权限隔离，不能注入到高等级的进程中。

注册表：

简单，但只能注入GUI进程

输入法：

复杂，但可注入任何接受输入法的进程。

*转载请注明来自游戏安全实验室（GSLAB.QQ.COM）

分享到：

Windows用户模式下注入方式总结

注入技术在病毒木马、游戏、打补丁等编程中应用很广泛，学习该技术不仅能帮助理解Windows工作原理，还能对病毒木马技术手段有更加深刻的理解，下面我们了解下各种注入方式吧。

一.DLL注入

在注入技术中，DLL注入是最常见最有效的技术。作者通常把代码编写成一个动态链接库，然后把这个动态链接库加载到正常进程中去，因为任何一个进程在加载动态链接库时都要执行DLL文件里入口函数DllMain中的代码，这样代码才得以执行。

1.修改注册表

修改 HKEY_LOCAL_MACHINE/Software/Microsoft/WindowsNT/CurrentVersion/Windows/AppInit_DLLs 的键值，改为待注入DLL的路径，这样一来，程序运行后只要加载了user32.dll，都会加载我们写入的DLL。

2.全局消息钩子注入

Windows应用程序是基于消息驱动的，任何线程只要注册窗口类都会有一个消息队列用于接收用户输入的消息和系统消息。为了拦截消息，Windows提出了钩子的概念。钩子（Hook）是Windows消息处理机制中的一个监视点，钩子提供一个回调函数。当在某个程序中安装钩子后，它将监视该程序的消息，在指定消息还没到达窗口之前钩子程序先捕获这个消息。这样就有机会对此消息进行过滤，或者对Windows消息实现监控。 消息钩子分为局部钩子和全局钩子。局部钩子是指仅拦截某一个进程的消息，全局钩子将拦截系统中所有进程的消息。

该技术需要我们写一个简单的加载该DLL的程序，使其调用DLL的钩子函数，该函数会下一个全局消息钩子，强迫所有发生该消息的进程加载该DLL，这样就实现了DLL注入。

3.手工修改导入表

该方法需要我们熟悉PE结构，找到导入表位置在其后面添加一个新节，写入待注入的DLL，并将该DLL的导出函数的一些信息写到正确的位置，若导出函数过多将花费很多时间在手工修改上，并且可能出现空间不足写入的情况。

4.远程线程注入DLL

通过利用2000/XP等系统所有进程加载Kernel32.dll的模块基址一致的特点，先使用OpenProcess函数打开远程进程的句柄，再用VirtualAllocEx和WriteProcessMemory将待注入DLL路径写入目标进程，找到LoadLibraryW的地址，最后用CreateRemoteThread使目标进程调用LoadLibraryW加载DLL，参数为写入的DLL路径。

5.APC注入

APC注入的原理是利用当线程被唤醒时APC中的注册函数会被执行的机制，并以此去执行我们的DLL加载代码，进而完成DLL注入的目的。利用QueueUserAPC()可以向APC队列投入Loadlibrary函数指针完成注入，其实这种方法配合CreateProcess使用注入最为简单，先挂起打开线程，再QueueUserAPC()，再恢复线程，完成注入。

6.DLL劫持

自己实现应用程序的某个DLL，完成其导出函数，并将其放于应用程序目录下，当程序打开并加载DLL时会优先加载该目录下的DLL，原来的DLL若在该目录下降其移走即可。

7.输入法注入

利用输入法在工作时需要向进程中加载Ime文件（其实就是个Dll）,我们构造自己的Ime文件，在Ime文件注入对方进程的时候加载我们自己的DLL完成注入

二.代码注入

对于病毒来讲，虽然远程注入DLL相对隐蔽，但是因为在被注入进程中多了个DLL模块，所以也没有做到完全隐蔽，而且也很容易将病毒清除。所以对于那些更高级的病毒则使用了更加隐蔽，更加难以清除的方法：远程代码注入。

这是指，在远程进程中开辟足够大的内存空间，然后将要执行的代码直接写过去，最后使用远程线程的方法运行注入代码。这样做可以使其他进程执行病毒代码，而又不会在对方进程中产生新的模块，从而使病毒能够更隐蔽地运行。

1.手工修改执行起始地址

使用二进制编辑工具打开目标程序，找到合适的地方写入二进制代码，将程序执行起始地址(AddressOfEntryPoint)改为注入代码处，在执行完后跳到源地址。

2.挂起线程注入

OpenThread-->SuspendThread-->申请内存-->写入代码-->GetThreadContext-->获取EIP-->修改EIP-->SetThreadContext-->ResumeThread。

3.挂起进程注入

CreateProcess注入方法之一，CREATE_SUSPENDED以挂起的方式打开进程，后面方法与挂起线程注入相似。

4.调试器注入

CreateProcess注入方法之二，DEBUG_ONLY_THIS_PROCESS以调试的方法打开进程，利用CREATE_PROCESS_DEBUG_EVENT，向目标程序中写入我们的ShellCode完成相应功能，并且我们的ShellCode中写入以CC断点，代码执行指令时触发EXCEPTION_DEBUG_EVENT事件，在EXCEPTION_DEBUG_EVENT的处理函数中回到原来的执行流程。

参考书籍：《计算机病毒揭秘与对抗》

参考网址：https://bbs.pediy.com/thread-217722.htm

posted @ 2018-04-09 18:52 仁札 阅读(702) 评论(0) 收藏 举报